

Robot Localization

EECS 116/216 Fall 2026 | Project 1

UC Berkeley

Due: October 12, 11:59 PM

Complete the project in **groups of 2–3**. Look up your group in the group assignments spreadsheet:

https://docs.google.com/spreadsheets/d/1fFw8qZSYLBhzKQAUNdIUVMYnrN_-dJR81FWVp9i4jwo

1 Overview

It's time to bring together everything you've learned in this course so far!

Implement an extended Kalman filter (EKF) to estimate the position, orientation, velocity, and gyroscope and accelerometer biases of a micro aerial vehicle. The goal is to infer unmeasured states and biases by fusing IMU readings with partial mocap observations. After the supplied initialization, use only the mocap channels specified for each part; all other channels are for comparison.

2 Sensor Data

The `data/` folder contains `studentdata1.mat`, `studentdata4.mat`, and `studentdata9.mat`. The provided Python loader returns the following fields; IMU and mocap samples occupy columns:

<code>time</code>	# Sample timestamps, seconds
<code>angular_velocity</code>	# IMU body-frame angular velocity, rad/s
<code>acceleration</code>	# IMU body-frame specific force, m/s ²
<code>vicon</code>	# Mocap system measurements, ordered as below

Each column of `vicon` contains

$$[x, y, z, \text{roll}, \text{pitch}, \text{yaw}, v_x, v_y, v_z, \omega_x, \omega_y, \omega_z]^T.$$

Position and velocity are expressed in the world frame, in metres and metres per second. Angles are in radians and angular velocity is in body coordinates, in radians per second. Camera images and tag detections are not used.

Use right-handed frames with world $+z$ upward and gravity $\mathbf{g} = [0, 0, -9.8]^T \text{ m/s}^2$. For this exercise, model the body, IMU, and tracked mocap frames as coincident, including their origins. The accelerometer measures *specific force*, not kinematic acceleration: a stationary, unbiased sensor aligned with the world frame reads $[0, 0, 9.8]^T \text{ m/s}^2$.

Use intrinsic ZYX rotations: yaw about z , then pitch about the rotated y , then roll about the twice-rotated x :

$$R = R_z(\text{yaw})R_y(\text{pitch})R_x(\text{roll}), \quad \mathbf{u}_{\text{world}} = R\mathbf{u}_{\text{body}}.$$

The stored angle order is **roll, pitch, yaw**; zero angles mean the body and world axes coincide. Gyroscope readings are angular velocities, not Euler-angle rates. Keep state angles unwrapped; wrap orientation innovations to $[-\pi, \pi)$.

The loader pairs equal recorded timestamps. In these three files, every IMU packet is marked available and has exactly one matching mocap sample in the same order. Use the resulting timestamps and all loaded samples in both parts. Treat paired records as simultaneous for this exercise. Initialize at the first timestamp. For each later sample, use forward Euler prediction with the previous sample's IMU readings over the elapsed interval, then update with the current mocap measurement.

3 Kalman Filter

Derive a shared process model and two measurement models:

- **Part 1**

Control inputs: Gyroscope and accelerometer measurements.

Measurement inputs: Mocap position and orientation (`vicon[0:6]`).

Estimated state: All 15 components: world-frame position, body-to-world orientation, world-frame velocity, and body-frame gyroscope and accelerometer biases (three components each).

- **Part 2**

Control inputs: Gyroscope and accelerometer measurements.

Measurement inputs: Mocap world-frame velocity only (`vicon[6:9]`).

Estimated state: All 15 components: world-frame position, body-to-world orientation, world-frame velocity, and body-frame gyroscope and accelerometer biases (three components each).

Use the state order listed above: `state[3]` is roll, `state[4]` is pitch, and `state[5]` is yaw (zero-based indices, as in `vicon`). The covariance follows the same order. Both parts start from the first mocap pose and velocity with zero biases; Part 2 uses pose only for initialization.

Complete `prediction.py` and `updates.py`; in `filter.py`, complete `noise_covariances()` and the loop (predict, update, save posterior estimates). Use only the supplied imports (NumPy, SciPy's `Rotation`, and project helpers); **no other dependencies**. You may add helpers in the prediction and update files. Preserve initialization, function signatures, return types, and all other code and data. Changes outside these areas receive zero points.

Choose and tune your own noise covariances in `noise_covariances()`: process noise Q is 12×12 (ordering documented in `predict()`); measurement noise R is 6×6 in Part 1 and 3×3 in Part 2. Your filter and step functions must use the matrices passed as inputs. For the numerical equivalence check, we supply our own Q and R ; use your chosen values for the report and explain your tuning.

First, set up your instructional account at <https://acropolis.cs.berkeley.edu/~account/webacct/>. Each group member should create their own instructional account. One member should create a **private GitHub repository** and invite the others so the group can collaborate and access its code outside the lab.

Use Python 3.10 on the Ubuntu 22.04 lab machines. In the project folder, run the following commands, reopening the terminal after the first command:

```
/share/instsw/anaconda3/bin/conda init
conda create -n eeecs116 python=3.10
conda activate eeecs116
python -m pip install -r requirements.txt
```

Run both parts for each of datasets 1, 4, and 9, for example:

```
python main.py --part 1 --dataset 9
python main.py --part 2 --dataset 9
```

4 Report

Submit a PDF report prepared with LaTeX or Word, at most **8 pages including plots**. List all group members' names and UC Berkeley student IDs at the top. Explain your process and measurement models and the reasoning behind your implementation. Include the supplied plots for both parts on all three datasets and discuss agreement with the mocap measurements. Some plotted measurements are also used by the filter, so agreement with them is not an independent accuracy evaluation.

5 Grade Policy and Submission

The total is **100 points**: Part 1 (60), Part 2 (30), and report quality and readability (10). Each call to `run_filter()` must finish within one minute per dataset on the lab machines, measured in wall-clock time and excluding data loading and plotting. Runs exceeding this limit will not be graded.

Put `report.pdf` alongside `main.py` in your completed project folder. Submit one archive named `submission.zip` with exactly this structure:

```
submission.zip
|-- report.pdf
|-- main.py
|-- filter.py
|-- prediction.py
|-- updates.py
|-- support.py
|-- requirements.txt
`-- data/
    |-- studentdata1.mat
    |-- studentdata4.mat
    `-- studentdata9.mat
```

The report and Python files must be **directly at the ZIP's top level**, not inside an enclosing `student/` or group folder. Keep filenames exactly as shown and include the supplied data unchanged. Do not include your Git repository, Python environment, cache files, or separate plot files; include the plots in `report.pdf`.

From the project folder containing `main.py` and `report.pdf`, run:

```
python -m zipfile -c ../submission.zip report.pdf main.py \
    filter.py prediction.py updates.py support.py requirements.txt data
python -m zipfile -l ../submission.zip
```

The first command creates `submission.zip` in the parent folder (replacing any previous archive with that name). The second lists its contents; check that they match the structure above. After extraction, the code must run using the commands above without moving files or changing paths.

Upload `submission.zip` to bCourses. **Each group member must submit the same ZIP using their own account.**